

Reliable Image Classification with Confidence Rejection

First submission: implementation and evaluation proposal

Introduction to Computer Vision | 2 October 2026

Team: Telman3000, LeoVesinML, Mysteri0K1ng, MedvAx-AI

Repository: [MedvAx-AI](#) / [project repository](#)

This proposal specifies the work we plan to carry out. The settings below are proposed defaults; numerical results will be reported after implementation and evaluation.

1. Project scope

We will build a ten-class image classifier that returns **REJECT** when confidence is below a selected threshold. Our research question is: how much can rejection reduce error among accepted predictions, what fraction of images must be deferred, and does calibration improve confidence quality on clean and degraded images?

The task is closed-set CIFAR-10 classification. Classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck [1]. Rejection is an abstention decision, not an extra ground-truth class. We will use one ImageNet-pretrained ResNet-18 with a trained linear head and compare decision policies on identical images. The scope is one dataset, one backbone, temperature scaling, and three corruption families; large-scale training and out-of-distribution detection are outside the planned experiment.

2. Dataset preparation

CIFAR-10 contains 60,000 RGB images at 32 x 32 resolution, with official partitions of 50,000 training and 10,000 test images [1]. We will download the official data through torchvision and use a class-balanced subset of 17,000 images. With seed 42, we will shuffle original indices within each class, save the manifests, and allocate these disjoint subsets:

Subset	Images	Per class	Use
Training	10,000	1,000	Fit the ten-class head
Model validation	2,000	200	Choose the head checkpoint
Calibration	1,500	150	Fit temperature T
Policy validation	1,500	150	Select rejection thresholds
Final test	2,000	200	Report final performance

The first four subsets come from the official training partition; the final test comes from the official test partition. The 5,000 held-out development images separate checkpoint, calibration and threshold decisions. Unselected images remain unused. We will check source-ID disjointness and keep every corrupted version in its source image's partition; final test data will not guide training or tuning.

We will convert images to RGB and apply the named pretrained weight transforms: bilinear resize to 256, center crop to 224, scale to [0,1], and normalize by ImageNet means (0.485, 0.456, 0.406) and standard deviations (0.229, 0.224, 0.225) [2]. Independent corruptions will be applied at native resolution before these transforms: Gaussian blur radii 0.5/1.0/1.5 pixels; Gaussian noise standard deviations 0.03/0.06/0.12 in [0,1], clipped and rounded to uint8; and JPEG qualities 70/40/15 with fixed encoder settings. Noise seeds will derive deterministically from source ID and severity. We will inspect development samples before freezing these settings.

3. Experimental plan

We will load ResNet18_Weights.IMAGENET1K_V1, freeze the backbone in evaluation mode, cache its features, and train only a new ten-class linear head [2]. Proposed training settings are cross-entropy, AdamW, learning rate 0.001, weight decay 0.0001, feature batch size 256, at most 30 epochs, and seed 42. The checkpoint with lowest model-validation negative log-likelihood will be selected, with earliest epoch breaking ties. We will first save the raw-confidence baseline, then reuse that checkpoint for all comparisons.

Condition	Confidence	Output policy
A: baseline	Raw softmax probabilities	Always predict a class
B: rejection	Raw softmax probabilities	Accept if confidence >= threshold
C: calibration	Temperature-scaled probabilities	Always predict a class
D: full method	Temperature-scaled probabilities	Accept if confidence >= threshold

3. Experimental plan (continued)

For logits z , confidence is the largest probability. We will fit a positive temperature T on the calibration split by minimizing negative log-likelihood using $\text{softmax}(z/T)$ [3]. A versus B isolates rejection; A versus C isolates calibration. Positive T preserves predicted labels and full-set accuracy. B and D will be compared at common thresholds and through risk-coverage curves, disclosing different accepted populations.

Separate raw/calibrated thresholds will be selected on clean policy-validation data over 0.00-1.00 in steps of 0.01. Target: empirical accepted accuracy $\geq 90\%$ and coverage $\geq 20\%$; maximize feasible coverage, breaking ties by smaller threshold. If infeasible, report this and use a labeled 0.80 fallback. Temperature and thresholds will be frozen before final testing and reused under corruption. The validation target does not guarantee test accuracy.

Evaluate A-D on clean images and nine corruption settings: **40 result rows**, each with 2,000 images. Metrics: full-set top-1 accuracy, accepted accuracy, coverage, rejection rate, selective risk (1 - accepted accuracy), negative log-likelihood and expected calibration error (15 equal-width bins). Accepted accuracy will accompany coverage/counts. Calibration metrics use all images, including rejected ones; accepted accuracy/risk are undefined when none are accepted.

The threshold sweep supplies the sensitivity experiment through descriptive risk-coverage curves; test outcomes will not select policies. Save comparison tables, reliability diagrams and per-image evidence. Explain at least three real failures, seeking confidently wrong acceptance, rejection of a correct prediction, and corruption-induced errors. If a type is absent, disclose it and substitute another observed failure. Compare images/confidences to support likely-cause hypotheses and limitations.

4. Implementation plan

We will use Python, PyTorch/torchvision, NumPy, Pillow, pandas and matplotlib; targeted checks will use pytest. LeoVesinML will pin a tested environment. The pipeline will:

1. Download data and named pretrained weights; save checksums, class order and split manifests.
2. Implement deterministic preprocessing/corruptions and verify leakage and sample alignment.
3. Extract features, train the head and save the baseline checkpoint and predictions.
4. Fit temperature, validate metrics and choose thresholds on their designated development subsets.
5. Freeze configuration, evaluate the complete test matrix and threshold sensitivity, then save metrics, figures and failure evidence.
6. Verify a fresh-environment run from exact README commands, and prepare a local demo showing predictions, confidence, threshold and rejection.

Record seeds, software/hardware, configuration/checkpoint identifiers and actual extraction/training/inference times, separating full inference from cached-policy evaluation. Relative sprint: Days 1-2 data/interfaces; Days 3-4 baseline/policy freeze; Day 5 experiments; Days 6-7 reproduction/report/demo; Day 8 repair buffer. Confirm course deadlines before scheduling.

5. Team responsibilities

Team member	Main work	Report and validation contribution
Telman3000	Download, splits, leakage checks and blur/noise/JPEG generation	Dataset section; inspect failures and sample alignment
LeoVesinML	Environment, pretrained backbone, head training, baseline and runtime	Method/reproduction sections; review calibration isolation
MysteriOK1ng	Calibration, rejection policy, metric checks and threshold sensitivity	Experimental interpretation; review split integrity
MedvAx-AI	Evaluation runner, figures, failure analysis and local demo	Integrate team-written sections; independently verify reproduction

All members will review work, record actual contributions and prepare for individual Q&A. Final deliverables: reproducible code/environment, saved results, a two-page results summary with at least three explained failures, and a five-minute demo with at most three slides. Clarify the ten-minute session heading; provisionally reserve remaining time for Q&A.

References

1. Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*; [official CIFAR-10 dataset page](#).
2. PyTorch. [Torchvision ResNet-18 pretrained weights and transforms](#).
3. Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). [On Calibration of Modern Neural Networks](#). ICML, PMLR 70:1321-1330.